Distributed Simulation Based Search

Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Problem Description

Modern multiplayer games must create matches that are:

- **Fair** i.e players of similar skill and rank
- **Fast** i.e minimal wait time
- **Stable** i.e low latency, same region when possible

Core Problem:
How can we form balanced, high-quality matches quickly at large scale?

Traditional matchmaking systems (like matchtrees or simple queues) fail to balance these goals effectively in a large scale distributed system.

Project Idea

Distribute systems (players) **across server workloads** (game servers) **using Approximate Nearest Neighbor** (ANN) **and matchmaking**

- ANN- using vectorized attributes, select shard(s) likely to contain value
  - Represented by global regions
- Matchmaking- create “ideal” groups of players which will perform some work
  - MatchTree

Project Description

Question:

- **Can Approximate Nearest Neighbor (ANN) algorithms** like **FAISS** improve matchmaking quality in multiplayer games by creating **more balanced and faster matches** compared to traditional matchmaking algorithms?

Solution:

- **ANN-based matchmaking** will improve the quality of player matchups in terms of skill, rank, and latency.
- **ANN** will create **faster, more balanced matches** compared to baseline.

Measured Metrics:

Rationale for Metrics:

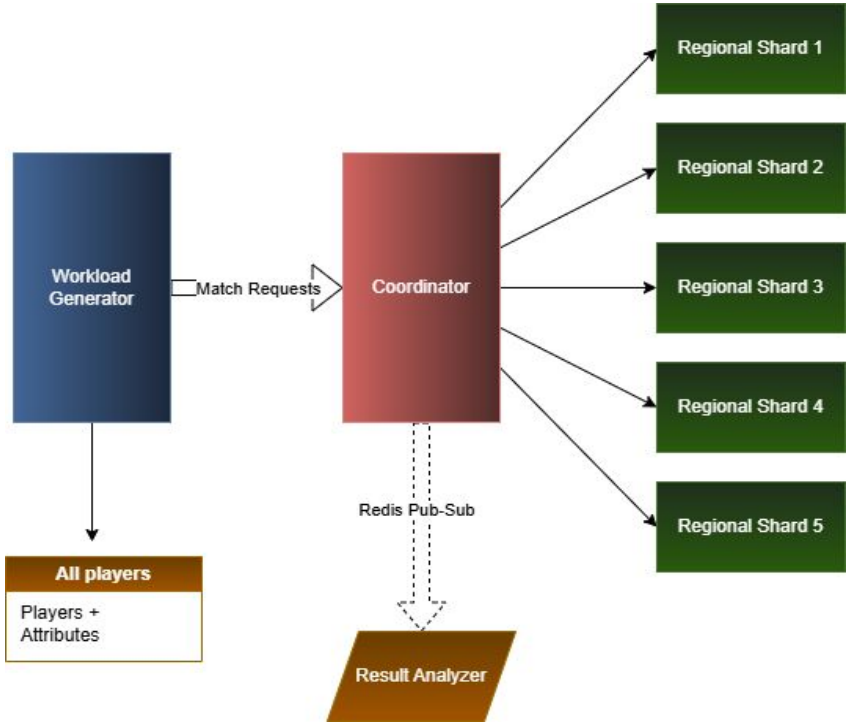
1. **Quality of Matches**: Use **z-scores** to quantify similarity between player attributes.
2. **Throughput**: Track the number of matches generated over time.
3. **Scalability**: Measure system performance as the number of players increases.

Baseline:

Creating matches on first 10 players as a first-come-first-serve basis.

System Overview

- Workload generator
- Coordinator
- Regional server(local shard)
- Result Analyzer
- Get Players for a match is the distributed operation.



Fariba Amiri, Sumaiya Azad, **Shreeda Ghatpande**, Jacob Hopkins & Andy Herbert

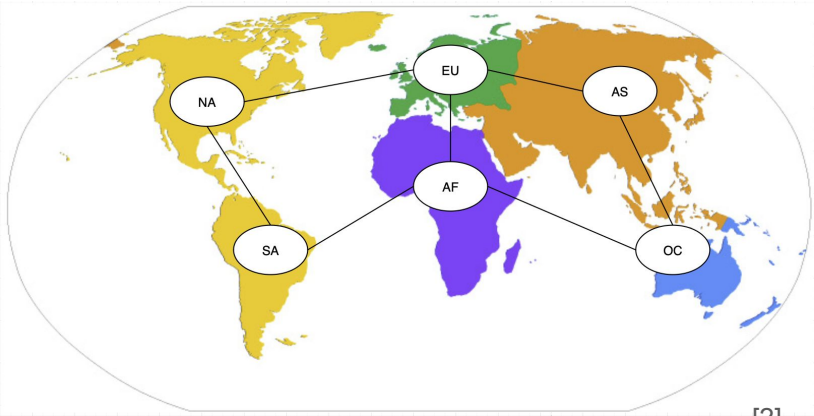
Coordinator

The ordering or queueing of players to be added to matches

Describe player attributes and encoding.

How coordinator creates queues

for different benchmarking?



[2]

MatchTree- matchmaking algorithm

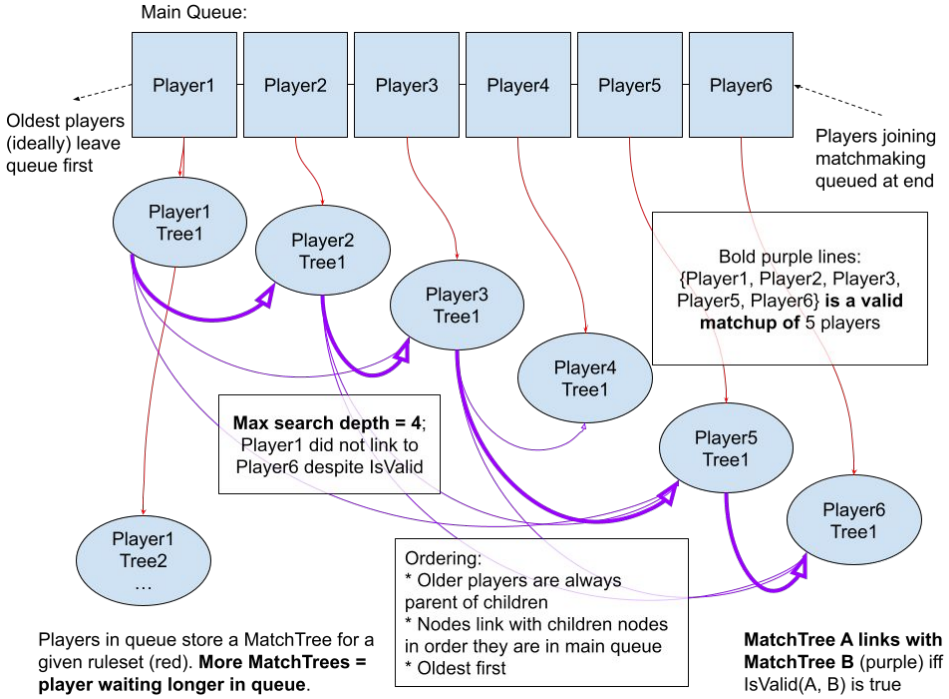
Emphasis: minimize wait time, maximize ideal matches

Ruleset: conditions which indicate if a group of players is valid

MatchTree: links to players creating valid matches with a ruleset

Handle thousands of players: pruning, optimizations

Priority further selects ideal matches



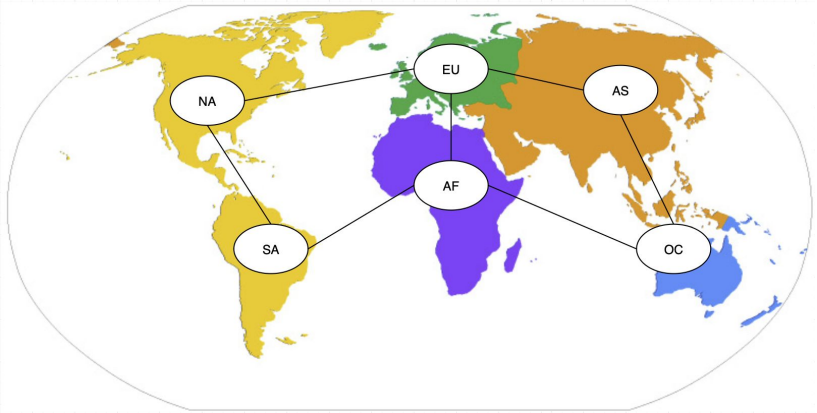
Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Player Generation and Attributes

Goal

- generate players with realistic gaming attributes
- simulate real online games by differing attributes

Attributes



- **Skill Level** - measure of how good the player is
  - mostly average players, few very high-skill players
- **Latency** - player's network delay to the server
  - mostly low latency, some high-latency players
- **Region** - player's geographical server representation
  - cross region matches increases delay
- **Rank** - based directly on Skill Level thresholds
  - determined from skill (Bronze → Silver → Gold → Diamond → Grandmaster)
- **Playtime Hours** - player's online time
  - some players play a lot and some very little

Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Player Encoding for FAISS

- For **Unweighted Encoding (Baseline FAISS)**, each player is converted to a numeric vector of
  - Skill
  - Latency
  - Rank Index
  - Playtime hour
  - One hot region vector

➡️ All attributes treated **equally** in distance calculation

- For **Weighted Encoding (Improved FAISS)**, we applied custom weights
  - Skill * 5
  - Latency * 2
  - Rank Index * 10
  - Playtime hour * 0.5
  - One hot region vector * 0.2

➡️ Makes matchmaking **prefer skill-rank similarity** more strongly

Workload Generation

Goal - simulate a realistic stream of **players requesting matches** and entering the queue

How it works

- Each request picks a player ID from CSV
- Probability of choosing a player is proportional to: `(playtime_hours + 1)^hot_player_bias`
- Active player appearing more often in matchmaking simulates real **online player behaviour**
- `hot_player_bias = 1.5` → heavy, active players appear **more often**

Outcome - A workload list of player IDs which simulates a **busy server** with **repeated match requests**

Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Matchmaking

- **FIFO Queue** Matchmaking
  - Maintain a queue of players arriving from workload
  - When there are enough distinct players in the queue,
    - form a match, mark them as busy
    - remove from the queue and continue

➡️ Simple, fast
➡️ only arrival order matters
➡️ extremely **low-quality** matches

- **FAISS-Based** Matchmaking
  - Neighbors selected using simple euclidean distance
  - For each workload seed:
    - picks its nearest non-busy neighbors

➡️ Distance purely based on raw attributes
➡️ Produces **moderate-quality** matches

- **Weighted FAISS** Matchmaking
  - Much better grouping of:
    - similar skill, rank & latency

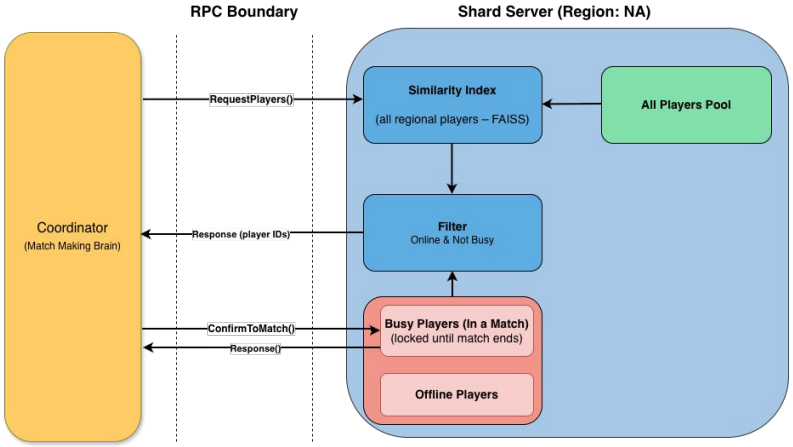
➡️ Simulates an intelligent, optimized coordinator
➡️ Produces **higher-quality, fairer** matches

Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Server Shard: What It Is

Server Shard (Regional Matchmaking Engine)

- Loads all players for **one region** (e.g., NA)
- Converts players into vectors
- Builds a FAISS index
- Manages a subset of players marked **online**
- Keeps track of players locked in a match (**busy**)
- Answers similarity queries from the coordinator
- Exposes gRPC APIs: `RequestPlayers` and `ConfirmToMatch`

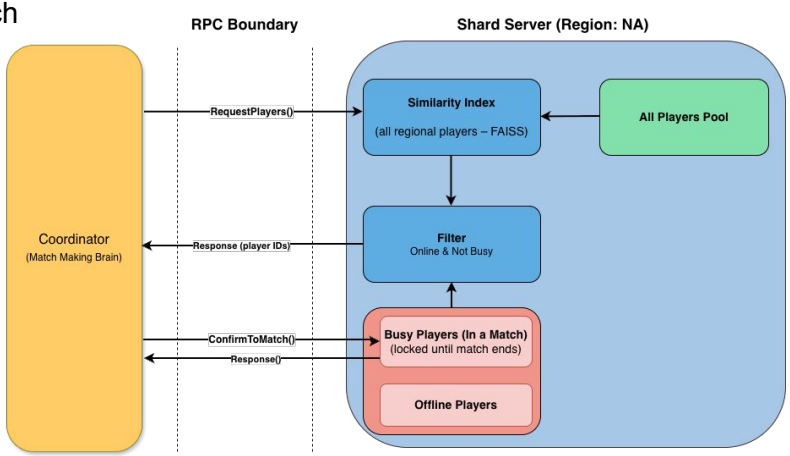


Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Server Shard: What It Does

Runtime Behavior

- Receives a query vector → performs FAISS nearest-neighbor search
- Filters results based on:
  - **Online** status
  - **Not busy** (not already matched)
- Returns best available players for that region
- Supports locking:
  - `ConfirmToMatch()` marks selected players as **busy**
- Runs a background process that **reshuffles online players** to simulate logins/logouts



Fariba Amiri, Sumaiya Azad, Shreeda Ghatpande, Jacob Hopkins & Andy Herbert

Results / Analysis

How do we Measure Quality?

What attributes are important?

FIFO baseline

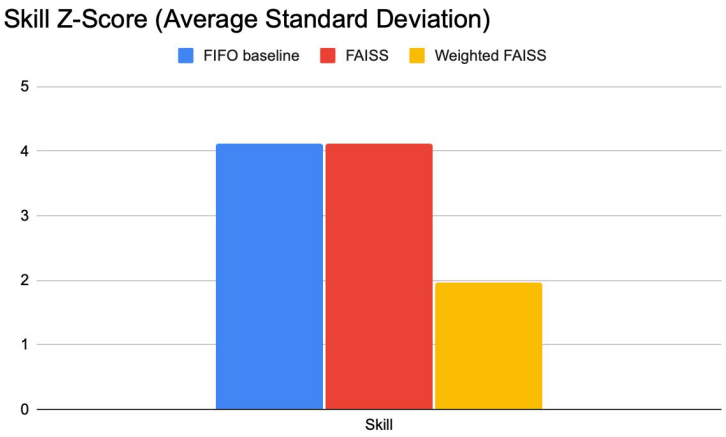
Score

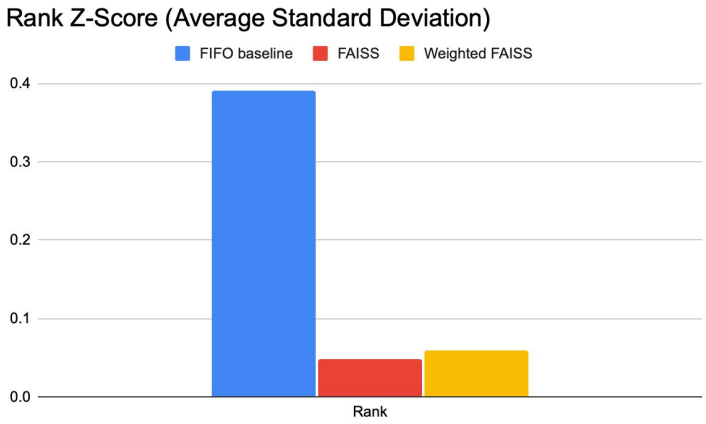
Mean

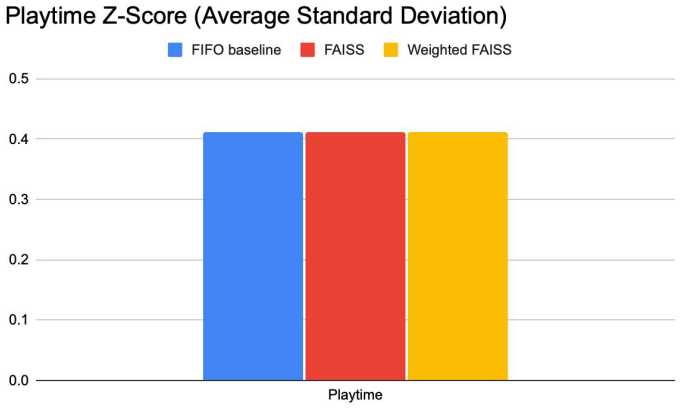
$$Z = \frac{x - \mu}{\sigma}$$

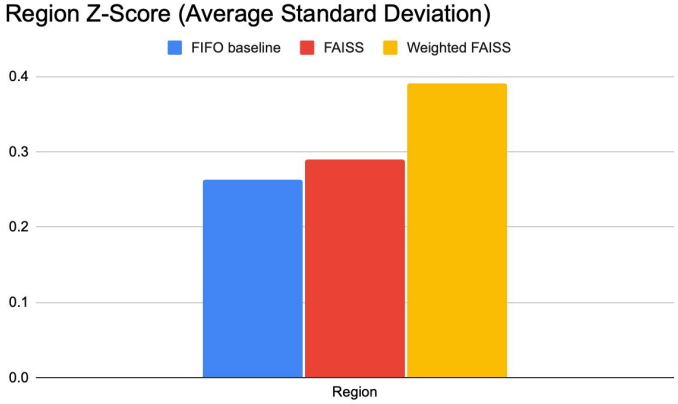
SD

[1]









Lower = Better

Takeaways / lessons learned

- Dataset generation and the impacts on the result
- Define the scope and components very early.
- Do not reinvent the wheel. We ended up drawing solutions from PA1 and PA2.
- Have small test code every component easier to debug.

Team Contributions

Shreeda - Coordinator and FAISS

Fariba - Regional servers

Sumaiya - Benchmarking matchmaking and dataset generation

Jacob - Server and Benchmarking

Andy - Matchmaking algorithm

Q & A

Credits

[1]

“What Is A Z-Score And How To Calculate Them?” Accessed: Nov. 29, 2025. [Online]. Available: https://toptipbio.com/z-score/

[2]

“World Map / World Atlas / Atlas of the World Including Geography Facts and Flags - WorldAtlas.com,” WorldAtlas. Accessed: Nov. 29, 2025. [Online]. Available: https://www.worldatlas.com/